UNIVERSIDAD TECNOLÓGICA DEL PERÚ / INGENIERÍA

GUÍA DE LABORATORIO 10

Integración del back-end seguro con Spring Boot

JPA + JPQL + Transacciones + Spring Security + JWT + APF2

JPA / MySQL	JPQL	@Transactional	Roles	JWT	APF2
Persistencia	Consultas	Consistencia	Autorización	Autenticación	Integración

Meta de la semana: demostrar que el back-end funciona como una solución integrada, segura y consistente.

CONTINUIDAD CURRICULAR
Semana 6: persistencia JPA/Hibernate → Semana 7: JPQL y transacciones → Semana 8: roles y permisos → Semana 9: autenticación JWT → Semana 10: integración, estabilización y evidencia APF2.

Versiones de referencia: Java 25 · Spring Boot 4.1.1 · Spring Security 7.1.1 · Maven · MySQL · VS Code · Postman.

1. Orientación del laboratorio

1.1. Logro de aprendizaje

Al finalizar la práctica, el estudiante integra los componentes desarrollados durante la Unidad 2 y demuestra el funcionamiento de un back-end RESTful persistente, transaccional y seguro, aplicando JPA/Hibernate, consultas JPQL, control transaccional, roles y permisos, y autenticación JWT.

> **ENFOQUE DE LA SEMANA 10**
> Esta semana no agrega una tecnología nueva. El reto es comprobar que las piezas implementadas en semanas anteriores funcionan juntas, con contratos HTTP coherentes, seguridad verificable, datos persistentes y comportamiento transaccional correcto.

1.2. Evidencias esperadas

- La aplicación inicia sin errores, conecta con MySQL y ejecuta correctamente las pruebas automatizadas existentes.
- POST /api/auth/login emite un JWT válido para usuarios existentes.
- Los endpoints de consulta aceptan ROLE_USER y ROLE_ADMIN según la política definida.
- Las operaciones de escritura y administración se restringen a ROLE_ADMIN.
- Al menos una consulta JPQL devuelve datos reales desde MySQL.
- Una operación de inventario ejecuta dos cambios dentro de una misma transacción y demuestra rollback ante un error.
- Se comprueban respuestas HTTP 200/201, 400/404/409, 401 y 403 con Postman.
- El proyecto se empaqueta satisfactoriamente con Maven y queda listo para la entrega APF2.

1.3. Distribución sugerida

Bloque	Actividad
A. Preparación y diagnóstico	Verificar proyecto, base de datos, perfiles, secretos y build.
B. Integración funcional	JPQL + transacción + endpoints protegidos + reporte administrativo.
C. Pruebas end-to-end	Login, JWT, roles, consulta, escritura, rollback y errores.
D. Calidad y APF2	mvn test/package, evidencias, README, matriz y checklist final.
E. Sustentación breve	Explicar arquitectura, decisiones y resultados observados.

2. Mapa de integración de la Unidad 2

Semana 6	Semana 7	Semana 8	Semana 9	Semana 10
JPA + CRUD	JPQL + TX	Roles	JWT	Integración

La arquitectura acumulada puede resumirse en el siguiente recorrido de una solicitud autenticada:

Arquitectura lógica integrada

```
Cliente / Postman
    │  Authorization: Bearer <JWT>
    ▼
SecurityFilterChain + JwtDecoder
    │  Authentication + Authorities (ROLE_USER / ROLE_ADMIN)
    ▼
@RestController
    ▼
@Service  ← aquí viven reglas de negocio y @Transactional
    ▼
JpaRepository + consultas derivadas / JPQL
    ▼
Hibernate / JPA
    ▼
MySQL
```

LECTURA CORRECTA
Spring Security protege la entrada HTTP; el Controller no debe contener lógica de autenticación. La capa Service coordina el negocio y las transacciones. El Repository se dedica a persistencia. Esta separación reduce acoplamiento y facilita las pruebas.

3. Caso integrador: API segura de inventario

Continuaremos con la API de productos e inventario construida desde la Semana 6. El sistema deberá permitir consultar productos, buscar mediante JPQL, registrar ajustes de stock de manera transaccional y exponer un resumen administrativo protegido por rol.

3.1. Política funcional y de seguridad

Operación	Método y ruta	Acceso esperado
Iniciar sesión	POST /api/auth/login	Público
Consultar perfil	GET /api/auth/me	JWT válido
Listar productos	GET /api/productos	USER o ADMIN
Buscar por rango	GET /api/productos/rango?min=&max=	USER o ADMIN
Crear producto	POST /api/productos	ADMIN
Actualizar producto	PUT /api/productos/{id}	ADMIN
Eliminar producto	DELETE /api/productos/{id}	ADMIN
Registrar salida	POST /api/inventario/{id}/salida	ADMIN
Resumen administrativo	GET /api/admin/resumen	ADMIN

3.2. Regla transaccional que vamos a comprobar

Una salida de inventario debe ejecutar dos operaciones como una sola unidad: (1) disminuir el stock del producto y (2) insertar un registro en movimientos_stock. Si cualquiera falla, ambos cambios deben revertirse.

```
INICIO TRANSACCIÓN
  1. Buscar producto
  2. Validar cantidad > 0
  3. Validar stock suficiente
  4. Descontar stock
  5. Registrar MovimientoStock tipo SALIDA
COMMIT

Si ocurre una excepción antes del COMMIT → ROLLBACK
```

4. Paso 1 - Abrir y diagnosticar el proyecto existente

1	**Abrir en VS Code el mismo proyecto de la Semana 9** No crees otro proyecto desde Spring Initializr. La finalidad es integrar y estabilizar el back-end existente.
2	**Verificar Java y Maven** Ejecuta java -version y mvn -version. Esta guía usa Java 25 y Maven compatible con Spring Boot 4.1.x.

Terminal de Visual Studio Code

```
java -version
mvn -version
```

3	**Verificar MySQL** Asegúrate de que el servicio esté iniciado y que tienda_db contenga los datos de productos, usuarios, roles y movimientos.

MySQL Workbench

```
USE tienda_db;
SHOW TABLES;
SELECT COUNT(*) AS productos FROM productos;
SELECT COUNT(*) AS usuarios FROM usuarios;
SELECT COUNT(*) AS roles FROM roles;
SELECT COUNT(*) AS movimientos FROM movimientos_stock;
```

4	**Ejecutar pruebas antes de modificar** Una integración profesional comienza con una línea base: debemos saber si el proyecto estaba estable antes de tocarlo.

```
mvn clean test
```

PUNTO DE CONTROL
Si el proyecto no compila o las pruebas fallan, corrige primero esos errores. No continúes agregando funcionalidades sobre una línea base inestable.

5. Paso 2 - Revisar configuración y dependencias

5.1. Dependencias Maven mínimas

El proyecto debe conservar los starters y módulos incorporados durante las semanas 6 a 9. Si usas el parent/BOM de Spring Boot, no fijes versiones individuales de Spring Security salvo necesidad justificada.

pom.xml - verificación de dependencias

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

```
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
</dependency>
```

5.2. Configuración de aplicación

src/main/resources/application.properties

```
spring.application.name=tienda

spring.datasource.url=jdbc:mysql://localhost:3306/tienda_db
spring.datasource.username=root
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

app.jwt.issuer=utp-desarrollo-web
app.jwt.ttl-minutes=30
app.jwt.secret-base64=${JWT_SECRET_BASE64}
```

> **SEGURIDAD MÍNIMA**
> No coloques una contraseña real de MySQL ni la clave JWT directamente en Git. Para el laboratorio, cárgalas mediante variables de entorno en la terminal desde la cual arrancas Spring Boot.

6. Paso 3 - Consolidar una consulta JPQL

La Semana 10 debe demostrar que la persistencia no se limita a CRUD. Conserva al menos una consulta JPQL real y úsala desde el flujo end-to-end.

ProductoRepository.java

```
package com.utp.tienda.repository;

import java.math.BigDecimal;
import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import com.utp.tienda.model.Producto;

public interface ProductoRepository extends JpaRepository<Producto, Long> {

    @Query("""
        SELECT p
        FROM Producto p
        WHERE p.precio BETWEEN :min AND :max
        ORDER BY p.precio ASC
    """)
    List<Producto> buscarPorRangoPrecio(
            @Param("min") BigDecimal min,
            @Param("max") BigDecimal max);

    long countByStockLessThan(int stock);
}
```

6.1. ¿Por qué esto demuestra JPQL?

- La consulta usa el nombre de la entidad Producto y sus atributos Java; no consulta directamente la tabla productos.
- Los parámetros :min y :max se vinculan con @Param, evitando concatenar texto SQL.
- La consulta queda encapsulada en Repository y puede reutilizarse desde Service.

7. Paso 4 - Confirmar la operación transaccional de inventario

Si tu Semana 7 ya implementó registrarSalida, no crees otro método. Compáralo con la versión de referencia y conserva la variante que respete las mismas reglas de negocio.

InventarioService.java

```
package com.utp.tienda.service;

import java.time.LocalDateTime;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.utp.tienda.model.MovimientoStock;
import com.utp.tienda.model.Producto;
import com.utp.tienda.repository.MovimientoStockRepository;
import com.utp.tienda.repository.ProductoRepository;

@Service
public class InventarioService {

    private final ProductoRepository productoRepository;
    private final MovimientoStockRepository movimientoRepository;

    public InventarioService(ProductoRepository productoRepository,
                             MovimientoStockRepository movimientoRepository) {
        this.productoRepository = productoRepository;
        this.movimientoRepository = movimientoRepository;
    }

    @Transactional
    public Producto registrarSalida(Long productoId, int cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad debe ser mayor que cero");
        }

        Producto producto = productoRepository.findById(productoId)
                .orElseThrow(() -> new ProductoNoEncontradoException(productoId));

        if (producto.getStock() < cantidad) {
            throw new StockInsuficienteException(
                    "Stock actual: " + producto.getStock() +
                    ", solicitado: " + cantidad);
        }

        producto.setStock(producto.getStock() - cantidad);

        MovimientoStock movimiento = new MovimientoStock();
        movimiento.setProducto(producto);
        movimiento.setTipo("SALIDA");
        movimiento.setCantidad(cantidad);
        movimiento.setFecha(LocalDateTime.now());
        movimientoRepository.save(movimiento);

        return producto;
    }
}
```

OBSERVA EL DIRTY CHECKING

El producto fue cargado dentro de la transacción y queda administrado por JPA. Al modificar stock, Hibernate puede generar el UPDATE al sincronizar la transacción. El movimiento sí se inserta explícitamente con save().

7.1. Endpoint de inventario

Fragmento de InventarioController.java

```
@PostMapping("/api/inventario/{id}/salida")
public ResponseEntity<Producto> registrarSalida(
        @PathVariable Long id,
        @RequestParam int cantidad) {

    return ResponseEntity.ok(inventarioService.registrarSalida(id, cantidad));
}
```

8. Paso 5 - Incorporar un resumen administrativo integrado

Crearemos una funcionalidad pequeña que obligue a recorrer varias capas sin introducir un nuevo tema: un resumen para administración. El endpoint será accesible solo por ROLE_ADMIN y consultará datos persistidos.

8.1. Crear el DTO de respuesta

src/main/java/com/utp/tienda/dto/ResumenBackendResponse.java

```
package com.utp.tienda.dto;

public record ResumenBackendResponse(
        long totalProductos,
        long productosConStockBajo,
        long totalMovimientos,
        String estado) {
}
```

8.2. Crear el servicio de lectura

AdminResumenService.java

```
package com.utp.tienda.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.utp.tienda.dto.ResumenBackendResponse;
import com.utp.tienda.repository.MovimientoStockRepository;
import com.utp.tienda.repository.ProductoRepository;

@Service
public class AdminResumenService {

    private final ProductoRepository productoRepository;
    private final MovimientoStockRepository movimientoRepository;

    public AdminResumenService(ProductoRepository productoRepository,
                               MovimientoStockRepository movimientoRepository) {
        this.productoRepository = productoRepository;
        this.movimientoRepository = movimientoRepository;
    }

    @Transactional(readOnly = true)
    public ResumenBackendResponse obtenerResumen() {
        long total = productoRepository.count();
        long bajo = productoRepository.countByStockLessThan(5);
        long movimientos = movimientoRepository.count();

        return new ResumenBackendResponse(
                total,
                bajo,
                movimientos,
                "BACKEND_OPERATIVO");
    }
}
```

8.3. Exponer el endpoint

AdminController.java

```
package com.utp.tienda.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.utp.tienda.dto.ResumenBackendResponse;
import com.utp.tienda.service.AdminResumenService;

@RestController
@RequestMapping("/api/admin")
public class AdminController {

    private final AdminResumenService service;

    public AdminController(AdminResumenService service) {
```

```
        this.service = service;
    }

    @GetMapping("/resumen")
    public ResumenBackendResponse resumen() {
        return service.obtenerResumen();
    }
}
```

9. Paso 6 - Consolidar la política de seguridad

La configuración final debe conservar la autenticación JWT de la Semana 9 y expresar de forma clara qué rutas son públicas, cuáles son de consulta y cuáles son administrativas.

SecurityConfig.java - fragmento de política final

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .csrf(csrf -> csrf.disable())
        .sessionManagement(sm ->
            sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/login").permitAll()

            .requestMatchers(HttpMethod.GET, "/api/productos/**")
                .hasAnyRole("USER", "ADMIN")

            .requestMatchers("/api/inventario/**")
                .hasRole("ADMIN")

            .requestMatchers("/api/admin/**")
                .hasRole("ADMIN")

            .requestMatchers(HttpMethod.POST, "/api/productos/**")
                .hasRole("ADMIN")
            .requestMatchers(HttpMethod.PUT, "/api/productos/**")
                .hasRole("ADMIN")
            .requestMatchers(HttpMethod.DELETE, "/api/productos/**")
                .hasRole("ADMIN")

            .anyRequest().authenticated()
        )
        .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()))
        .build();
}
```

> **IMPORTANTE**
> El orden de las reglas importa. Las rutas específicas deben declararse antes de anyRequest().authenticated(). Mantén la conversión de claim roles → ROLE_USER / ROLE_ADMIN implementada en la Semana 9.

10. Paso 7 - Ejecutar la aplicación con variables de entorno

Usa la misma clave JWT que empleaste al generar los tokens. Si reinicias con otra clave, los tokens anteriores dejarán de validar.

10.1. Windows PowerShell

```
$env:DB_PASSWORD="tu_password_mysql"
$env:JWT_SECRET_BASE64="TU_CLAVE_BASE64_DE_LABORATORIO"
mvn spring-boot:run
```

10.2. Linux / macOS

```
export DB_PASSWORD='tu_password_mysql'
export JWT_SECRET_BASE64='TU_CLAVE_BASE64_DE_LABORATORIO'
mvn spring-boot:run
```

> **NO COMPARTAS SECRETOS**
> Las capturas para el APF2 no deben mostrar la contraseña de MySQL, la clave de firma ni el JWT completo. Puedes mostrar solo los primeros y últimos caracteres del token.

11. Paso 8 - Prueba end-to-end con Postman

11.1. Obtener token de USER

```
POST http://localhost:8080/api/auth/login
Content-Type: application/json

{
  "username": "usuario",
  "password": "Usuario123*"
}
```

Copia accessToken y configúralo en Authorization → Bearer Token.

11.2. Consultar productos con USER

```
GET http://localhost:8080/api/productos
Authorization: Bearer <TOKEN_USER>
```

Resultado esperado: 200 OK.

11.3. Intentar una operación ADMIN con USER

```
GET http://localhost:8080/api/admin/resumen
Authorization: Bearer <TOKEN_USER>
```

Resultado esperado: 403 Forbidden. El token es válido, pero no tiene la autoridad requerida.

11.4. Obtener token de ADMIN y consultar resumen

```
POST http://localhost:8080/api/auth/login
Content-Type: application/json

{
  "username": "admin",
  "password": "Admin123*"
}
```

```
GET http://localhost:8080/api/admin/resumen
Authorization: Bearer <TOKEN_ADMIN>
```

Ejemplo de respuesta

```
{
  "totalProductos": 5,
  "productosConStockBajo": 1,
  "totalMovimientos": 3,
  "estado": "BACKEND_OPERATIVO"
}
```

12. Paso 9 - Demostrar JPQL y la transacción

12.1. Ejecutar una consulta JPQL

```
GET http://localhost:8080/api/productos/rango?min=100&max=1000
Authorization: Bearer <TOKEN_USER>
```

Verifica en la consola el SQL generado por Hibernate y compáralo con la consulta JPQL del repositorio.

12.2. Registrar una salida exitosa

```
POST http://localhost:8080/api/inventario/1/salida?cantidad=2
Authorization: Bearer <TOKEN_ADMIN>
```

Comprueba en MySQL que disminuyó el stock y que apareció un movimiento de tipo SALIDA:

```
SELECT id, nombre, stock FROM productos WHERE id = 1;
SELECT id, tipo, cantidad, fecha, producto_id
FROM movimientos_stock
WHERE producto_id = 1
ORDER BY id DESC;
```

12.3. Forzar un rollback

Anota el stock y el número de movimientos. Luego intenta retirar una cantidad mayor al stock disponible:

```
POST http://localhost:8080/api/inventario/1/salida?cantidad=99999
Authorization: Bearer <TOKEN_ADMIN>
```

La API debe responder con un error de negocio (recomendado: 409 Conflict). Vuelve a ejecutar las consultas SQL. El stock y el número de movimientos deben permanecer iguales.

> **EVIDENCIA DECISIVA**
> El rollback no se demuestra solo mostrando un mensaje de error. Debes comparar el estado de la base de datos antes y después del fallo y evidenciar que no existe un cambio parcial.

13. Matriz de pruebas de integración

N.°	Escenario	Credencial / rol	Esperado
1	Login USER válido	Usuario + password	200 + accessToken
2	GET /api/productos sin token	Ninguna	401
3	GET /api/productos	ROLE_USER	200
4	POST /api/productos	ROLE_USER	403
5	POST /api/productos	ROLE_ADMIN	200/201
6	GET rango JPQL	ROLE_USER	200 + datos filtrados
7	POST salida válida	ROLE_ADMIN	200 + stock disminuye
8	POST salida con stock insuficiente	ROLE_ADMIN	409 + rollback
9	GET /api/admin/resumen	ROLE_USER	403
10	GET /api/admin/resumen	ROLE_ADMIN	200
11	Token alterado	JWT firma inválida	401
12	Producto inexistente	ROLE_ADMIN	404

13.1. 401 y 403: explicación que debes poder sustentar

Código	Qué significa	Ejemplo
401 Unauthorized	La solicitud no tiene una autenticación válida.	Sin JWT, JWT vencido o firma inválida.
403 Forbidden	La identidad sí fue autenticada, pero carece del permiso exigido.	ROLE_USER intenta abrir /api/admin/resumen.

14. Paso 10 - Prueba automatizada de autorización

La prueba siguiente valida el contrato de seguridad del endpoint administrativo sin depender de un token real emitido por /login. Spring Security Test crea una autenticación JWT simulada.

src/test/java/.../AdminControllerSecurityTest.java

```
package com.utp.tienda.controller;

import static org.mockito.Mockito.when;
import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.jwt;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import com.utp.tienda.dto.ResumenBackendResponse;
import com.utp.tienda.service.AdminResumenService;

@WebMvcTest(AdminController.class)
class AdminControllerSecurityTest {

    @Autowired
    MockMvc mockMvc;

    @MockitoBean
    AdminResumenService service;

    @Test
    void adminPuedeConsultarResumen() throws Exception {
        when(service.obtenerResumen())
            .thenReturn(new ResumenBackendResponse(5, 1, 3, "BACKEND_OPERATIVO"));

        mockMvc.perform(get("/api/admin/resumen")
                .with(jwt().authorities(
                    new SimpleGrantedAuthority("ROLE_ADMIN"))))
            .andExpect(status().isOk());
    }

    @Test
    void userNoPuedeConsultarResumen() throws Exception {
        mockMvc.perform(get("/api/admin/resumen")
                .with(jwt().authorities(
                    new SimpleGrantedAuthority("ROLE_USER"))))
            .andExpect(status().isForbidden());
    }
}
```

Ejecución

```
mvn test
```

> **SI TU TEST NO ARRANCA**
> Dependiendo de cómo importaste SecurityConfig y JwtDecoder en tu proyecto, el slice @WebMvcTest puede necesitar @Import(SecurityConfig.class) o mocks adicionales. Mantén el objetivo: probar 200 para ADMIN y 403 para USER.

15. Paso 11 - Quality Gate antes del APF2

Antes de entregar, el proyecto debe superar una revisión mínima similar a la que se realiza en un equipo profesional.

Control	Comando / evidencia	Criterio de aprobación
Compilación	mvn clean test	BUILD SUCCESS y pruebas aprobadas.
Empaquetado	mvn clean package	JAR generado en target/.
Persistencia	Consultas MySQL	Productos, usuarios, roles y movimientos coherentes.
JPQL	GET rango	Resultado filtrado correcto.
Transacción	Salida válida + fallo	Commit en éxito; rollback en error.
Autenticación	POST /api/auth/login	Token emitido solo con credenciales válidas.

Control	Comando / evidencia	Criterio de aprobación
Autorización	Matriz 401/403/200	Reglas por rol correctamente aplicadas.
Secretos	Revisión Git/archivos	Sin password ni JWT secret versionados.
Documentación	README + Postman	Proyecto reproducible por un tercero.

15.1. Empaquetar la aplicación

```
mvn clean package

# Resultado esperado:
# target/tienda-0.0.1-SNAPSHOT.jar
```

> **JAR NO SIGNIFICA DESPLIEGUE**
> El empaquetado demuestra que el back-end puede producir un artefacto ejecutable. El despliegue a servidor/nube se trabajará posteriormente en la Unidad 3.

16. Producto de la Semana 10: preparación del APF2

El sílabo ubica en la Semana 10 la integración de la Unidad 2 y el segundo avance del proyecto final. La siguiente estructura es una propuesta de evidencia técnica para que el equipo demuestre lo aprendido; no reemplaza una rúbrica oficial adicional que el docente pudiera indicar.

16.1. Entregables sugeridos

- Código fuente Maven compilable con estructura Controller → Service → Repository.
- Modelo JPA persistente en MySQL y script/datos mínimos para reproducir las pruebas.
- Al menos una consulta JPQL funcional y documentada.
- Al menos una operación @Transactional con evidencia de commit y rollback.
- Usuarios/roles persistentes; contraseñas codificadas con BCrypt.
- Login JWT y consumo de endpoints con Bearer Token.
- Matriz de pruebas Postman con 401, 403, 404/409 y 200/201.
- Pruebas automatizadas ejecutadas con mvn test.
- README con requisitos, variables de entorno, forma de ejecutar y endpoints principales.
- Diagrama simple de arquitectura y una breve explicación de las decisiones del equipo.

16.2. Estructura sugerida del README

```
# Nombre del proyecto

## 1. Objetivo
## 2. Arquitectura
## 3. Requisitos (Java, Maven, MySQL)
## 4. Variables de entorno
## 5. Base de datos
## 6. Ejecución
## 7. Endpoints
## 8. Usuarios/roles de prueba (sin contraseñas reales)
## 9. Pruebas
## 10. Evidencias de transacción y seguridad
```

17. Preguntas para reflexión

- ¿Por qué @Transactional debe colocarse en una operación de negocio que coordina varios cambios?
- ¿Qué diferencia práctica existe entre una consulta JPQL y una consulta SQL nativa?
- ¿Por qué un JWT válido puede terminar en 403?
- ¿Qué valida Spring Security antes de permitir que una solicitud llegue al Controller?
- ¿Qué ocurriría si se descontara stock y el insert del movimiento fallara sin una transacción?

- ¿Por qué la contraseña no debe aparecer dentro del JWT?
- ¿Qué ventaja aporta inyectar Repository/Service por constructor?
- ¿Qué evidencias permiten afirmar que el back-end está listo para pasar a la integración con Angular?

18. Errores frecuentes y diagnóstico

Síntoma	Causa probable	Qué revisar
401 en todos los endpoints	JWT ausente/expirado, clave distinta o issuer inválido.	Header Authorization, JWT_SECRET_BASE64, JwtDecoder.
403 con ADMIN	Roles no se convirtieron a ROLE_ADMIN o regla no coincide.	Claims roles, JwtAuthenticationConverter y orden de reglas.
400 en query de rango	Tipos/parámetros incorrectos.	Nombres min/max, BigDecimal y contrato del Controller.
500 al registrar salida	Excepción no manejada o relación JPA incorrecta.	Logs, @RestControllerAdvice, mappings y FK.
Stock cambia pero no hay movimiento	Operación no es atómica o excepción fue capturada.	@Transactional en Service y manejo de excepciones.
mvn test falla	Contexto de seguridad/JPA incompleto en test.	Slice usado, mocks/imports y dependencia spring-security-test.
Aplicación no arranca	DB/secret/puerto faltante.	MySQL, variables de entorno y puerto 8080.

19. Cierre de la Unidad 2

Al completar esta guía, el estudiante debe poder explicar y demostrar que una API empresarial no es únicamente un conjunto de endpoints. El back-end integra persistencia, consultas, reglas de negocio, transacciones, autenticación, autorización, manejo de errores y pruebas. Esa integración es la base técnica que permitirá, desde la Semana 11, conectar un frontend Angular sin trasladar al cliente responsabilidades que pertenecen al servidor.

> **RESULTADO ESPERADO**
> Back-end RESTful persistente + consultas JPQL + transacciones confiables + usuarios y roles + JWT + pruebas + artefacto JAR.

19.1. Checklist final del estudiante

- ☐ Puedo iniciar el proyecto desde VS Code sin errores.
- ☐ Puedo explicar Controller, Service, Repository y Entity.
- ☐ Puedo mostrar una consulta JPQL funcionando.
- ☐ Puedo demostrar commit y rollback con datos de MySQL.
- ☐ Puedo explicar la diferencia entre autenticación y autorización.
- ☐ Puedo obtener y usar un JWT Bearer.
- ☐ Puedo explicar 401 vs 403.
- ☐ mvn clean test y mvn clean package terminan correctamente.
- ☐ Mi repositorio no contiene secretos.
- ☐ Tengo evidencias claras para sustentar el APF2.

20. Referencias técnicas

- Spring Boot 4.1.1 - System Requirements: https://docs.spring.io/spring-boot/system-requirements.html
- Spring Security 7.1.1 - Reference: https://docs.spring.io/spring-security/reference/
- Spring Security - OAuth2 Resource Server JWT: https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html
- Spring Framework - Transaction Management: https://docs.spring.io/spring-framework/reference/data-access/transaction.html
- Spring Data JPA - Reference: https://docs.spring.io/spring-data/jpa/reference/